**BASH SCRIPT-02**

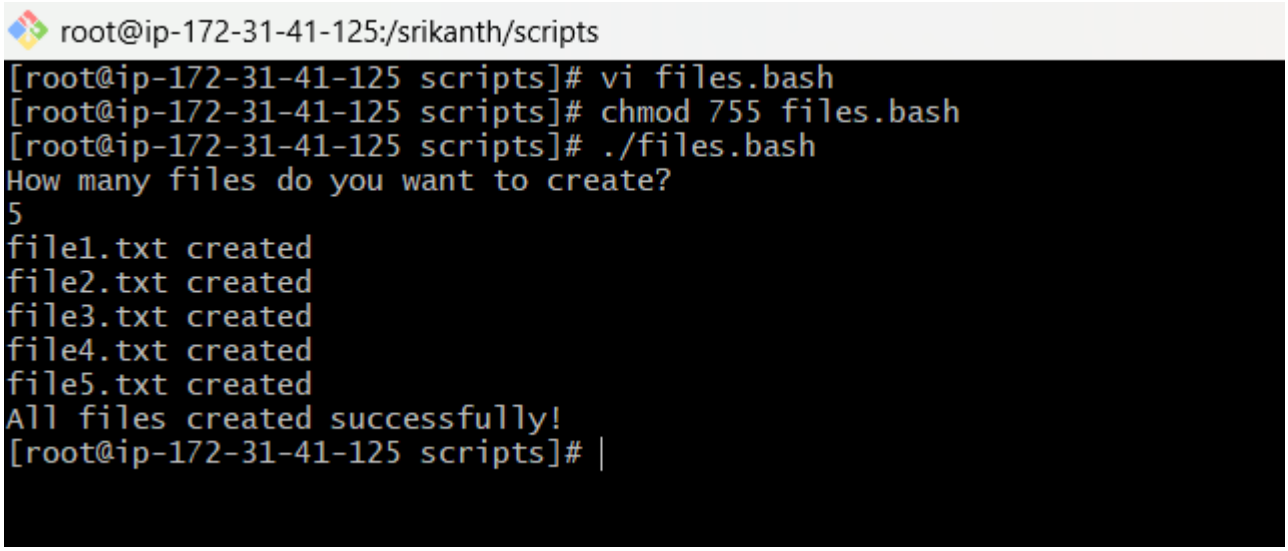
1.Create a bash script to check if a directory is available or not

- Connect to the EC2 instance using Git Bash.
- Create a Bash script named check_directory.sh.
- Write a script to accept a directory path from the user.
- Use the -d condition to check whether the directory exists.
- Display “Directory exists” if the directory is available.
- Display “Directory does not exist” if the directory is unavailable.
- Save the script and exit the editor.
- Give the script execution permission.
- Execute the script and verify the output.

```
[ec2-user@ip-172-31-41-125 ~]$ pwd
/home/ec2-user
[ec2-user@ip-172-31-41-125 ~]$ mkdir -p ~/bash_practice
[ec2-user@ip-172-31-41-125 ~]$ cd ~/bash_practice
[ec2-user@ip-172-31-41-125 bash_practice]$ nano check_directory.sh
[ec2-user@ip-172-31-41-125 bash_practice]$ chmod +x check_directory.sh
[ec2-user@ip-172-31-41-125 bash_practice]$ ./check_directory.sh
Enter directory path:
/c/Users
Directory does not exist.
[ec2-user@ip-172-31-41-125 bash_practice]$ ls -l check_directory.sh
-rwxr-xr-x. 1 ec2-user ec2-user 150 Oct 16 12:50 check_directory.sh
[ec2-user@ip-172-31-41-125 bash_practice]$ nano check_directory.sh
[ec2-user@ip-172-31-41-125 bash_practice]$ ./check_directory.sh
Enter directory path:
/Srikanth
Directory does not exist.
[ec2-user@ip-172-31-41-125 bash_practice]$ pwd
/home/ec2-user/bash_practice
[ec2-user@ip-172-31-41-125 bash_practice]$ ./check_directory.sh
Enter directory path:
/home
Directory exists.
[ec2-user@ip-172-31-41-125 bash_practice]$
```

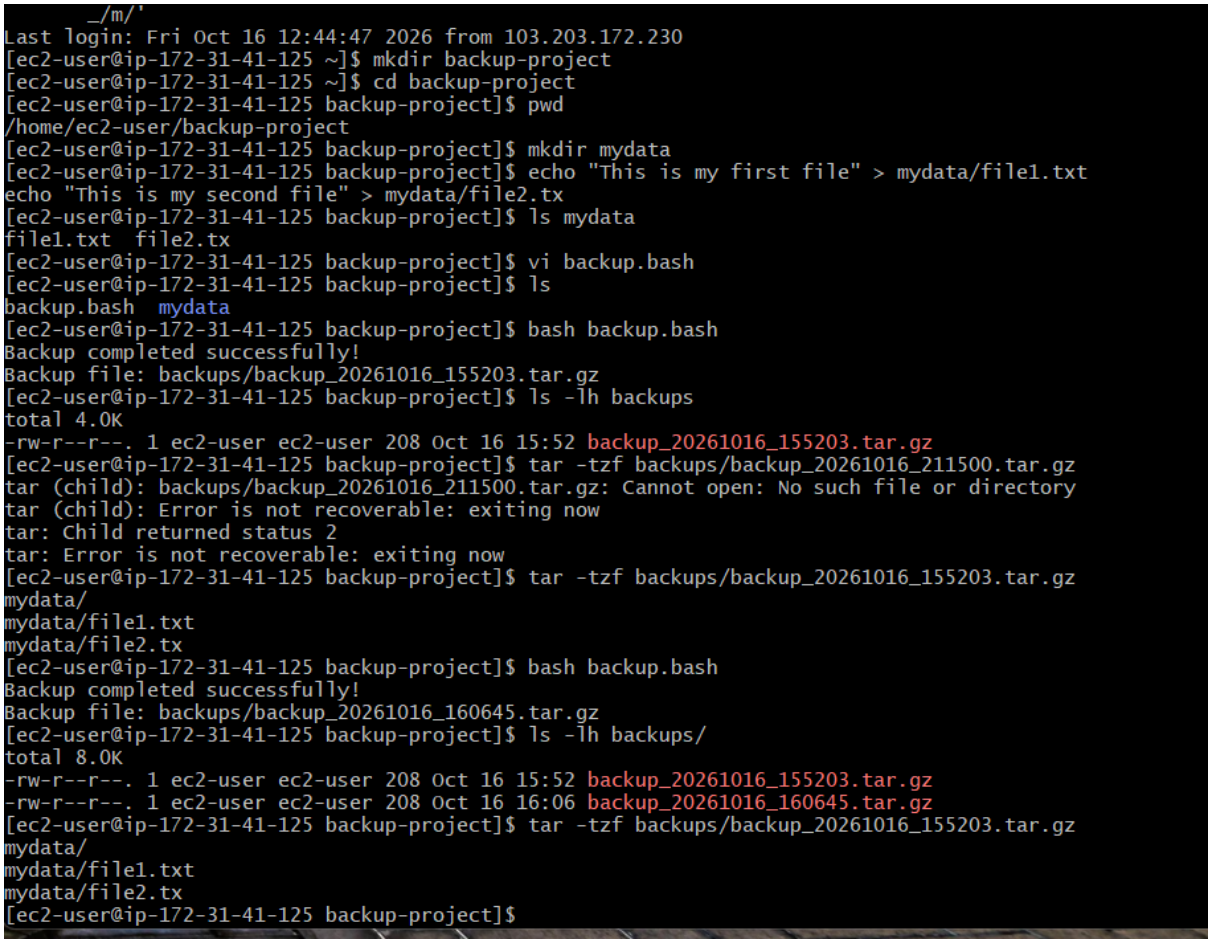
2.Create a bash script to **create multiple files**.

1. Connect to the EC2 instance using Git Bash.
2. Create a Bash script named create_files.sh.
3. Open the script in a text editor.
4. Write the script to create multiple files using a loop.
5. Specify the names of the files to be created.
6. Save the script and exit the editor.
7. Give the script execution permission.
8. Execute the script.
9. Verify that all files have been created successfully.



3. Create a bash script to **take a backup of a directory**.

- Connect to the EC2 instance using Git Bash.
- Create a Bash script named backup.sh.
- Open the script in a text editor.
- Specify the source directory to back up.
- Specify the destination directory to store the backup.
- Write the script to create a compressed backup using the tar command.
- Save the script and exit the editor.
- Give the script execution permission.
- Execute the script.
- Verify that the backup file has been created successfully.


_/m/'
Last login: Fri Oct 16 12:44:47 2026 from 103.203.172.230
[ec2-user@ip-172-31-41-125 ~]$ mkdir backup-project
[ec2-user@ip-172-31-41-125 ~]$ cd backup-project
[ec2-user@ip-172-31-41-125 backup-project]$ pwd
/home/ec2-user/backup-project
[ec2-user@ip-172-31-41-125 backup-project]$ mkdir mydata
[ec2-user@ip-172-31-41-125 backup-project]$ echo "This is my first file" > mydata/file1.txt
echo "This is my second file" > mydata/file2.tx
[ec2-user@ip-172-31-41-125 backup-project]$ ls mydata
file1.txt  file2.tx
[ec2-user@ip-172-31-41-125 backup-project]$ vi backup.bash
[ec2-user@ip-172-31-41-125 backup-project]$ ls
backup.bash  mydata
[ec2-user@ip-172-31-41-125 backup-project]$ bash backup.bash
Backup completed successfully!
Backup file: backups/backup_20261016_155203.tar.gz
[ec2-user@ip-172-31-41-125 backup-project]$ ls -lh backups
total 4.0K
-rw-r--r--. 1 ec2-user ec2-user 208 Oct 16 15:52 backup_20261016_155203.tar.gz
[ec2-user@ip-172-31-41-125 backup-project]$ tar -tzf backups/backup_20261016_211500.tar.gz
tar (child): backups/backup_20261016_211500.tar.gz: Cannot open: No such file or directory
tar (child): Error is not recoverable: exiting now
tar: Child returned status 2
tar: Error is not recoverable: exiting now
[ec2-user@ip-172-31-41-125 backup-project]$ tar -tzf backups/backup_20261016_155203.tar.gz
mydata/
mydata/file1.txt
mydata/file2.tx
[ec2-user@ip-172-31-41-125 backup-project]$ bash backup.bash
Backup completed successfully!
Backup file: backups/backup_20261016_160645.tar.gz
[ec2-user@ip-172-31-41-125 backup-project]$ ls -lh backups/
total 8.0K
-rw-r--r--. 1 ec2-user ec2-user 208 Oct 16 15:52 backup_20261016_155203.tar.gz
-rw-r--r--. 1 ec2-user ec2-user 208 Oct 16 16:06 backup_20261016_160645.tar.gz
[ec2-user@ip-172-31-41-125 backup-project]$ tar -tzf backups/backup_20261016_155203.tar.gz
mydata/
mydata/file1.txt
mydata/file2.tx
[ec2-user@ip-172-31-41-125 backup-project]$

4. Create a bash script to **install Nginx on an EC2 server**.

- Launch an EC2 instance in AWS.
- Connect to the EC2 instance using Git Bash.
- Create a Bash script named install_nginx.sh.
- Open the script in a text editor.
- Add commands to update the Ubuntu package repository.
- Add a command to install Nginx.
- Add commands to start and enable the Nginx service.
- Save the script and exit the editor.
- Give the script execution permission.
- Execute the script.
- Verify that the Nginx service is running.
- Open the EC2 public IP address in a browser to check the Nginx web page.

```
[ec2-user@ip-172-31-41-125 backup-project]$ mkdir nginx-project
[ec2-user@ip-172-31-41-125 backup-project]$ cd nginx-project
[ec2-user@ip-172-31-41-125 nginx-project]$ pwd
/home/ec2-user/backup-project/nginx-project
[ec2-user@ip-172-31-41-125 nginx-project]$ vi install_nginx.bash
[ec2-user@ip-172-31-41-125 nginx-project]$ chmod +x install_nginx.bash
[ec2-user@ip-172-31-41-125 nginx-project]$ ./install_nginx.sh
-bash: ./install_nginx.sh: No such file or directory
[ec2-user@ip-172-31-41-125 nginx-project]$ ls -l
total 4
-rwxr-xr-x. 1 ec2-user ec2-user 399 Oct 16 16:24 install_nginx.bash
[ec2-user@ip-172-31-41-125 nginx-project]$ bash install_nginx.bash
Starting Nginx installation...
Amazon Linux 2023 Kernel Livepatch repository                   805 kB/s |  89 kB     00:00
Dependencies resolved.
Nothing to do.
Complete!
Last metadata expiration check: 0:00:03 ago on Fri Oct 16 16:27:19 2026.
Dependencies resolved.
================================================================================
 Package                  Architecture   Version                      Repository          Size
================================================================================
Installing:
 nginx                    x86_64         1:1.30.5-1.amzn2023.0.1      amazonlinux          34 k
Installing dependencies:
 generic-logos-httpd      noarch         18.0.0-12.amzn2023.0.3       amazonlinux          19 k
 gperftools-libs          x86_64         2.9.1-1.amzn2023.0.3         amazonlinux         308 k
 libunwind                x86_64         1.4.0-5.amzn2023.0.3         amazonlinux          66 k
 nginx-core               x86_64         1:1.30.5-1.amzn2023.0.1      amazonlinux         711 k
 nginx-filesystem         noarch         1:1.30.5-1.amzn2023.0.1      amazonlinux          10 k
 nginx-mimetypes          noarch         2.1.49-3.amzn2023.0.3        amazonlinux          21 k

Transaction Summary
================================================================================
Install  7 Packages

Total download size: 1.1 M
Installed size: 3.8 M
Downloading Packages:
(1/7): generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch.rpm    789 kB/s |  19 kB     00:00
(2/7): libunwind-1.4.0-5.amzn2023.0.3.x86_64.rpm                2.3 MB/s |  66 kB     00:00
(3/7): gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64.rpm          8.1 MB/s | 308 kB     00:00
(4/7): nginx-1.30.5-1.amzn2023.0.1.x86_64.rpm                   1.2 MB/s |  34 kB     00:00
(5/7): nginx-core-1.30.5-1.amzn2023.0.1.x86_64.rpm               20 MB/s | 711 kB     00:00
(6/7): nginx-filesystem-1.30.5-1.amzn2023.0.1.noarch.rpm        356 kB/s |  10 kB     00:00
(7/7): nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch.rpm         879 kB/s |  21 kB     00:00
--------------------------------------------------------------------------------
Total                                                           8.1 MB/s | 1.1 MB     00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                        1/1
  Running scriptlet: nginx-filesystem-1:1.30.5-1.amzn2023.0.1.noarch       1/7
  Installing       : nginx-filesystem-1:1.30.5-1.amzn2023.0.1.noarch       1/7
  Installing       : nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch          2/7
  Installing       : libunwind-1.4.0-5.amzn2023.0.3.x86_64                 3/7
```



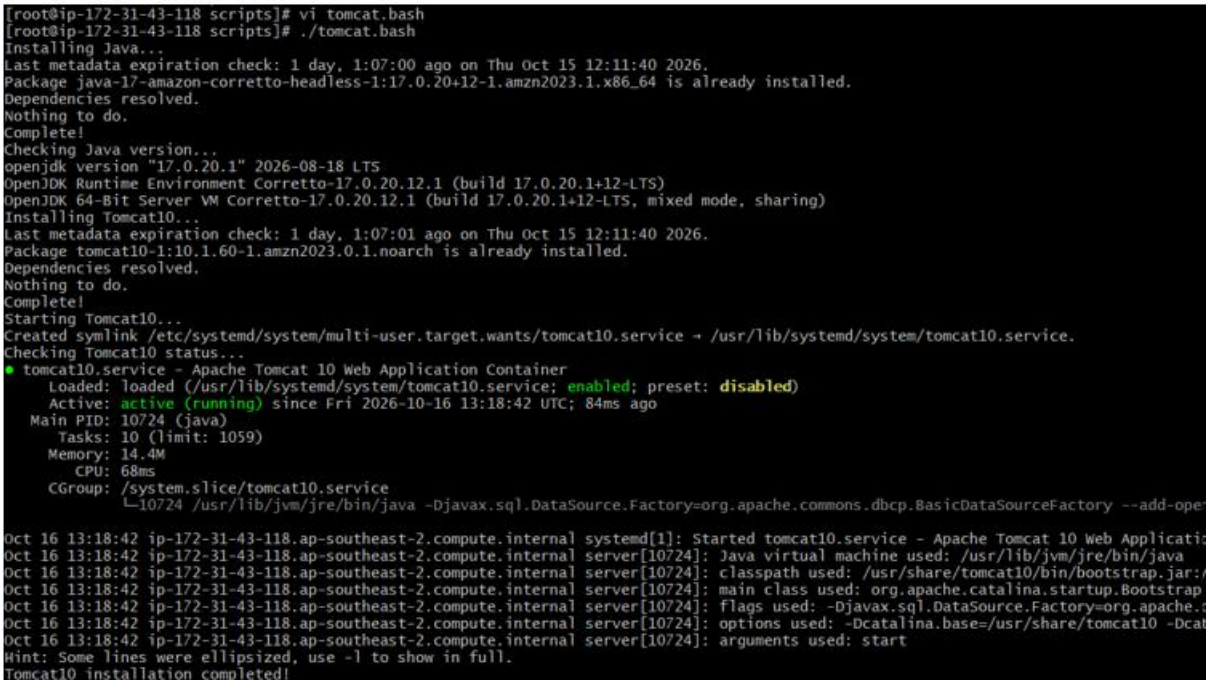

# Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

*Thank you for using nginx.*

5. Create a bash script to **install Apache Tomcat on an EC2 server**.



6. Create a bash script to **check if the Nginx service is running**, if not running then script should start the service.

```
Last login: Fri Oct 16 08:56:16 2026 from 103.203.172.230
[ec2-user@ip-172-31-43-118 ~]$ sudo su
[root@ip-172-31-43-118 ec2-user]# cd
[root@ip-172-31-43-118 ~]# cd /gopi/scripts
[root@ip-172-31-43-118 scripts]# vi niginxcheck.bash
[root@ip-172-31-43-118 scripts]# chmod 755 niginxcheck.bash
[root@ip-172-31-43-118 scripts]# mv niginxcheck.bash nginxcheck.bash
[root@ip-172-31-43-118 scripts]# chmod 755 nginxcheck.bash
[root@ip-172-31-43-118 scripts]# ./nginxcheck.bash
Nginx is already running.
[root@ip-172-31-43-118 scripts]# cat nginx.bash

#!/bin/bash

echo "Installing Nginx..."

sudo dnf install nginx -y

sudo systemctl enable nginx
sudo systemctl start nginx

echo "Nginx installation completed!"
sudo systemctl status nginx --no-pager

[root@ip-172-31-43-118 scripts]#
```

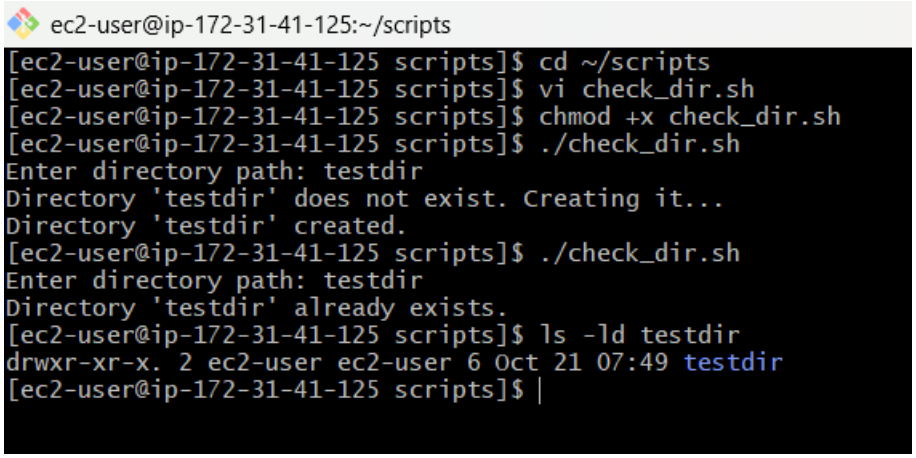
7. Create a bash script for a **calculator**.

- Connect to the EC2 instance using Git Bash.
- Create a Bash script named calculator.sh.
- Open the script in a text editor.
- Write the script to accept two numbers from the user.
- Display a menu of arithmetic operations: addition, subtraction, multiplication, and division.
- Accept the user's choice of operation.
- Use conditional statements to perform the selected calculation.
- display the calculation result.
- Save the script and exit the editor.
- Give the script execution permission.
- Execute the script and test the different operations.

```
A newer release of "Amazon Linux" is available.
  Version 2023.12.20261016:
Run "/usr/bin/dnf check-release-update" for full release and version update info
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Sat Oct 17 04:13:45 2026 from 103.203.172.230
[ec2-user@ip-172-31-41-125 ~]$
[ec2-user@ip-172-31-41-125 ~]$ mkdir -p ~/scripts && cd ~/scripts
[ec2-user@ip-172-31-41-125 scripts]$ vi calculator.sh
[ec2-user@ip-172-31-41-125 scripts]$ chmod +* calculater.sh
chmod: invalid mode: ‘+*’
Try 'chmod --help' for more information.
[ec2-user@ip-172-31-41-125 scripts]$ chmod +x calculater.sh
chmod: cannot access 'calculater.sh': No such file or directory
[ec2-user@ip-172-31-41-125 scripts]$ chmod +x calculator.sh
[ec2-user@ip-172-31-41-125 scripts]$ sudo dnf install -y bc
Last metadata expiration check: 0:22:47 ago on Wed Oct 21 07:17:24 2026.
Package bc-1.07.1-14.amzn2023.0.2.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-41-125 scripts]$ ./calculator.sh
Enter first number: 12
Enter second number: 4
Choose an operation:
1) Add
2) Subtract
3) Multiply
4) Divide
Enter choice [1-4]: 4
Result: 3.00
[ec2-user@ip-172-31-41-125 scripts]$
```

8. Create a bash script to **check if a directory exists**, if not then create a directory.

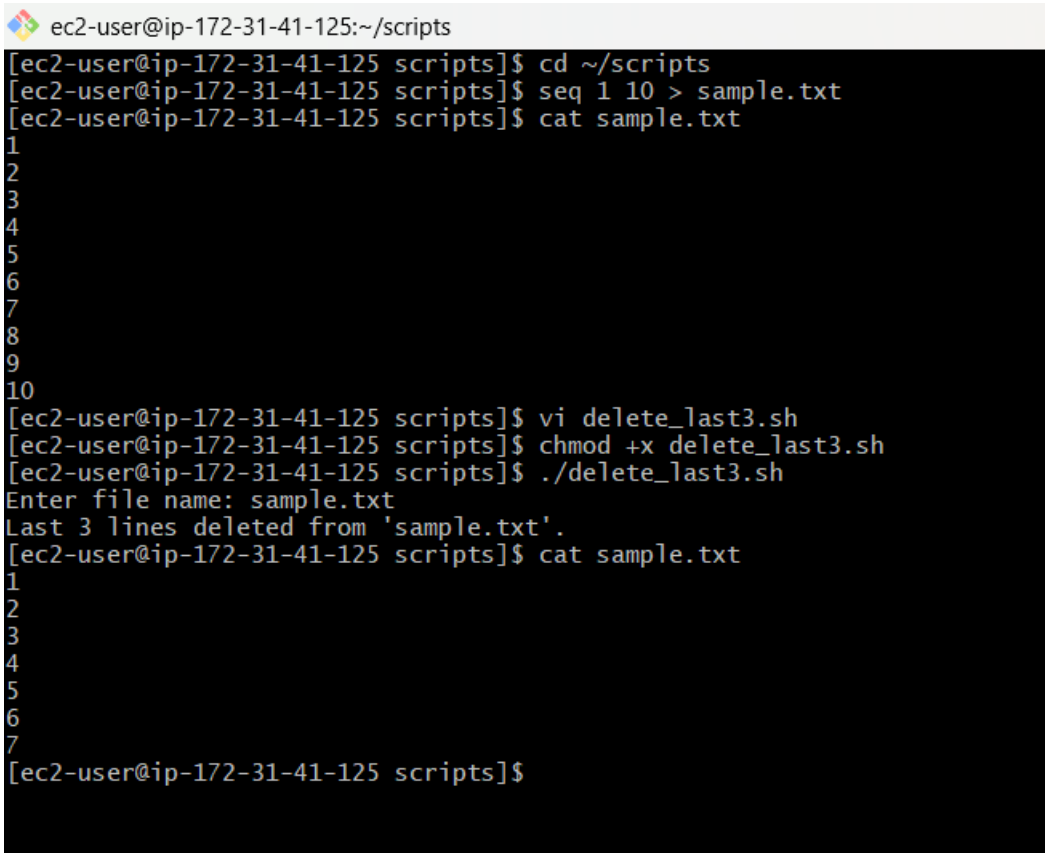
- Connect to the  EC2 instance using Git Bash.
- Create a Bash script named check_create_directory.sh.
- Open the script in a text editor.
- Specify the directory name or path.
- Use an if-else condition to check whether the directory exists.
- If the directory exists, display a message confirming it.
- If the directory does not exist, create it using the mkdir command.
- Save the script and exit the editor.
- Give the script execution permission.
- Execute the script and verify the output.



9. Create a bash script to **delete the last 3 lines of a file**.

- Connect to the  EC2 instance using Git Bash.
- Create a Bash script named delete_last3.sh.
- Open the script in a text editor.
- Specify the file name or path.
- Use the sed command to remove the last 3 lines of the file.
- Save the script and exit the editor.
- Give the script execution permission.
- Execute the script.

- Verify that the last 3 lines have been deleted successfully.



10. Bash script to monitor cpu and if it is more than 80% then send email notification.

- Connect to the EC2 instance using Git Bash.
- Install the required monitoring and email utilities.
- Create a Bash script named cpu_monitor.sh.
- Open the script in a text editor.
- Write the script to check the current CPU usage.
- Set the CPU usage threshold to 80%.
- Use an if condition to check whether CPU usage exceeds 80%.
- If CPU usage exceeds 80%, send an email notification to the specified email address.
- If CPU usage is below or equal to 80%, display a message indicating normal CPU usage.
- Save the script and exit the editor.
- Give the script execution permission.
- Execute the script and verify the output.
- Configure and test email delivery to ensure notifications can be received.
- Optionally, schedule the script using Cron to monitor CPU usage automatically at regular intervals.

```
[ec2-user@ip-172-31-41-125 scripts]$ vi ~/.mailrc
[ec2-user@ip-172-31-41-125 scripts]$ echo "SMTP test" | mail -s "EC2 test" mumma
disrikanth4@gmail.com
s-nail: New-style URL used without *v15-compat* being set
/home/ec2-user/dead.letter 3/50
s-nail: ... message not sent
[ec2-user@ip-172-31-41-125 scripts]$ vi ~/.mailrc
[ec2-user@ip-172-31-41-125 scripts]$ echo "SMTP test" | mail -s "EC2 test" mumma
disrikanth4@gmail.com
[ec2-user@ip-172-31-41-125 scripts]$
```

ec2-user@ip-172-31-41-125:~/scripts

```
ec2-user@ip-172-31-41-125 scripts]$ ./cpu_monitor.sh
urrent CPU usage: 50%
PU usage is normal.
ec2-user@ip-172-31-41-125 scripts]$ nproc
2
ec2-user@ip-172-31-41-125 scripts]$ yes > /dev/null &
2] 6491
ec2-user@ip-172-31-41-125 scripts]$ yes > /dev/null &
3] 6494
ec2-user@ip-172-31-41-125 scripts]$ sleep 5
/[ec2-user@ip-172-31-41-125 scripts]$ ./cpu_monitor.sh
urrent CPU usage: 100%
lert email sent.
ec2-user@ip-172-31-41-125 scripts]$ sed -i 's|^EMAIL=.*|EMAIL="mummadisrikanth4
gmail.com"|' ~/scripts/cpu_monitor.sh
ec2-user@ip-172-31-41-125 scripts]$ grep EMAIL= ~/scripts/cpu_monitor.sh
MAIL="mummadisrikanth4@gmail.com"
ec2-user@ip-172-31-41-125 scripts]$ yes > /dev/null &
4] 8059
ec2-user@ip-172-31-41-125 scripts]$ yes > /dev/null &
5] 8117
ec2-user@ip-172-31-41-125 scripts]$ sleep 5
ec2-user@ip-172-31-41-125 scripts]$ ./cpu_monitor.sh
urrent CPU usage: 100%
lert email sent.
ec2-user@ip-172-31-41-125 scripts]$ pkill yes
3]   Terminated              yes > /dev/null
1]   Terminated              yes > /dev/null
2]   Terminated              yes > /dev/null
4]-  Terminated              yes > /dev/null
ec2-user@ip-172-31-41-125 scripts]$ pgrep yes
5]+  Terminated              yes > /dev/null
ec2-user@ip-172-31-41-125 scripts]$ cat ~/scripts/cpu_monitor.sh
#!/bin/bash

THRESHOLD=80
EMAIL="mummadisrikanth4@gmail.com"

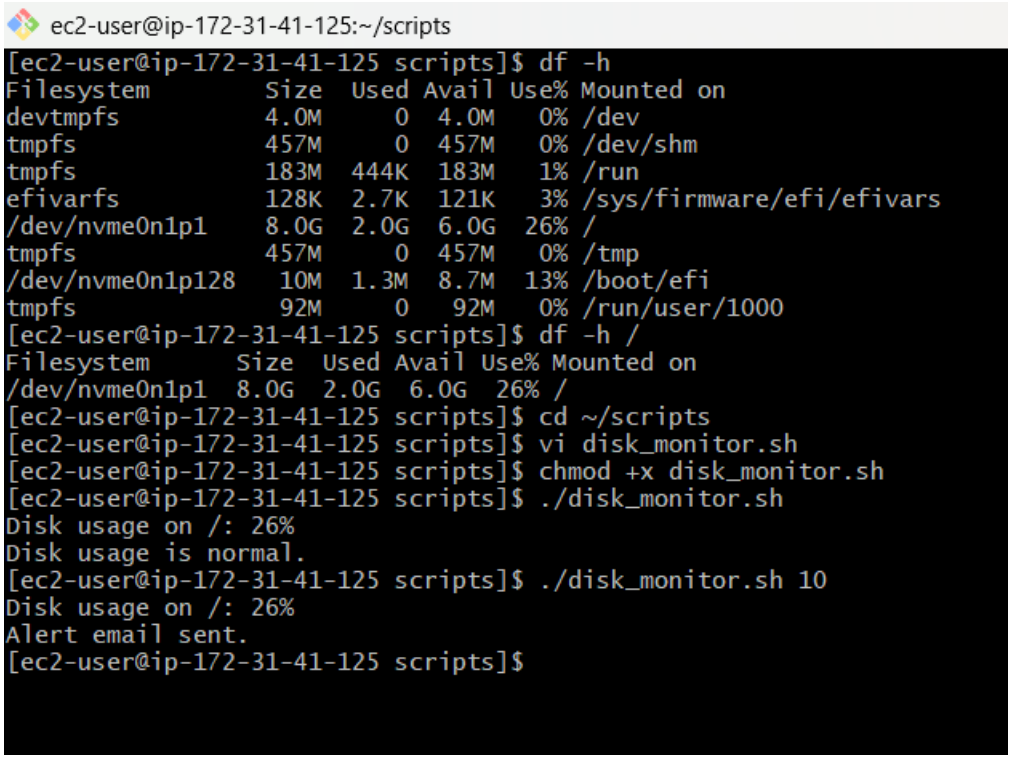
cpu=$(vmstat 1 2 | tail -1 | awk '{print 100 - $15}')
echo "Current CPU usage: ${cpu}%"

if [ "$cpu" -gt "$THRESHOLD" ]; then
  {
    echo "CPU usage on $(hostname) is ${cpu}% (threshold: ${THRESHOLD}%)"
    echo "Time: $(date)"
    echo
    echo "Top processes:"
    ps -eo pid,comm,%cpu --sort=-%cpu | head -6
  } | mail -s "ALERT: High CPU on $(hostname)" "$EMAIL"
  echo "Alert email sent."
else
  echo "CPU usage is normal."
fi

ec2-user@ip-172-31-41-125 scripts]$ ./cpu_monitor.sh
urrent CPU usage: 0%
PU usage is normal.
ec2-user@ip-172-31-41-125 scripts]$
```

# 11. Bash script to monitor disk space and if it is more than 80% then send email notification.

- Connect to the  EC2 instance using Git Bash.
- Install the required email utilities.
- Create a Bash script named `disk_monitor.sh`.
- Open the script in a text editor.
- Use the `df` command to check disk space usage.
- Set the disk usage threshold to 80%.
- Use an `if` condition to check whether disk usage exceeds 80%.
- If disk usage exceeds 80%, send an email notification to the specified email address.
- If disk usage is below or equal to 80%, display a message indicating normal disk usage.
- Save the script and exit the editor.
- Give the script execution permission.
- Execute the script and verify the output.
- Configure and test email delivery to ensure notifications can be received.
- Optionally, schedule the script using Cron to monitor disk space automatically at regular intervals.




ALERT: High disk usage on ip-172-31-41-125.ap-southeast-2.compute.internal Inbox ×

**mummadisrikanth4@gmail.com**
to me

Disk usage on ip-172-31-41-125.ap-southeast-2.compute.internal is 26% (threshold: 10%)
Partition: /
Time: Wed Oct 21 10:09:32 UTC 2026

Filesystem summary:
Filesystem Size Used Avail Use% Mounted on
/dev/nvme0n1p1 8.0G 2.0G 6.0G 26% /

Largest directories under /:
2.0G /
1.6G /usr
275M /var
52M /boot
22M /etc
21M /opt

12. Bash script to monitor memory and if it is more than 80% then send email notification.

 Connect to the Ubuntu EC2 instance using Git Bash.

 Install the required email utilities.

 Create a Bash script named memory_monitor.sh.

 Open the script in a text editor.

 Use the free command to check memory usage.

 Set the memory usage threshold to 80%.

 Use an if condition to check whether memory usage exceeds 80%.

 If memory usage exceeds 80%, send an email notification to the specified email address.

 If memory usage is below or equal to 80%, display a message indicating normal memory usage.

 Save the script and exit the editor.

 Give the script execution permission.

 Execute the script and verify the output.

 Configure and test email delivery to ensure notifications can be received.

 Optionally, schedule the script using Cron to monitor memory usage automatically at regular intervals.

ec2-user@ip-172-31-41-125:~/scripts

```
[ec2-user@ip-172-31-41-125 scripts]$ free -m
               total        used        free      shared  buff/cache   available
Mem:             913         217         276           0         419         535
Swap:              0           0           0
[ec2-user@ip-172-31-41-125 scripts]$ cd ~/scripts
[ec2-user@ip-172-31-41-125 scripts]$ vi memory_monitor.sh
[ec2-user@ip-172-31-41-125 scripts]$ chmod +x memory_monitor.sh
[ec2-user@ip-172-31-41-125 scripts]$ ./memory_monitor.sh
Memory usage: 42%
Memory usage is normal.
[ec2-user@ip-172-31-41-125 scripts]$ ./memory_monitor.sh 10
Memory usage: 42%
Alert email sent.
[ec2-user@ip-172-31-41-125 scripts]$ python3 -c "x = b'a' * (550 * 1024 * 1024);
 import time; time.sleep(40)" &
sleep 5
[1] 9422
[ec2-user@ip-172-31-41-125 scripts]$ ./memory_monitor.sh
Memory usage: 91%
Alert email sent.
[ec2-user@ip-172-31-41-125 scripts]$
```

ALERT: High memory usage on ip-172-31-41-125.ap-southeast-2.compute.internal

**mummadisrikanth4@gmail.com**
to me

Memory usage on ip-172-31-41-125.ap-southeast-2.compute.internal is 91% (threshold: 80%)
Time: Wed Oct 21 10:28:08 UTC 2026

Memory summary:

| | total | used | free | shared | buff/cache | available |
|---|---|---|---|---|---|---|
| Mem: | 913Mi | 700Mi | 83Mi | 0.0Ki | 128Mi | 80Mi |
| Swap: | 0B | 0B | 0B | | | |

Top memory-consuming processes:

| PID | COMMAND | %MEM |
|---|---|---|
| 9422 | python3 | 61.0 |
| 1953 | ssm-agent-worke | 2.4 |
| 1120 | systemd-journal | 2.1 |
| 1 | systemd | 1.5 |
| 1886 | amazon-ssm-agen | 1.4 |

**Exercise: Crontab Entries**

1. **April 5th Midnight**

   0 0 5 4 * /root/cronjob/april5.bash

2. **5th of Every November, January, June if it is a Thursday**

   **0 0 5 1,6,11 4 /root/cronjob/thursday.bash**

   At first, I thought my above answer is correct.

   **0 0 5 1,6,11 * [ "$(date +\%u)" -eq 4 ] && /root/cronjob/thursday.bash**

3. **At 05 and 27th minutes of 9, 10, 11 hours every day**

   5,27  9,10,11 * * * /root/cronjob/everydayjob.bash

4. **34 minutes of 9th hour on 15th August**

   34 9 15 8 * /root/cronjob/15august.bash

5. **Every midnight**

   0 0 * * * /root/cronjob/everymidnyt.bash

6. **Every Weekend (Saturday night 11:59 PM)**

   59 23 * * 6 /root/cronjob/weeklysatjob.bash

7. **After every reboot**

   @reboot /root/cronjob/rebootjob.bash